

Material Classification

Visual onboarding guide

Turn an orthophoto into a material map. Six classes, two very different sources of truth, and one orchestrator that fuses them. This deck is the picture version of **ONBOARDING.md** — read it front to back once, then keep the markdown open.

What this thing does

You give it a georeferenced aerial image. It gives back the same scene with every pixel recolored by what it is *made of* — plus a sidecar material table that a simulation toolchain reads.



The six materials

CLASS	COLOR	SOURCE	TYPICAL CONTENT
 BM_VEGETATION	#006400	KMEANS	Trees, grass, crops
 BM_SAND	#E6C882	KMEANS	Beach, dune, bare pale ground
 BM_SOIL	#55371E	KMEANS	Dirt, tilled earth
 BM ASPHALT	#2D2D30	MASK	Roads, paved surfaces
 BM_CONCRETE	#B4B4B4	MASK	Buildings, rooftops
 BM_WATER	#1C6BA0	MASK	Rivers, lakes, pools

What gets written

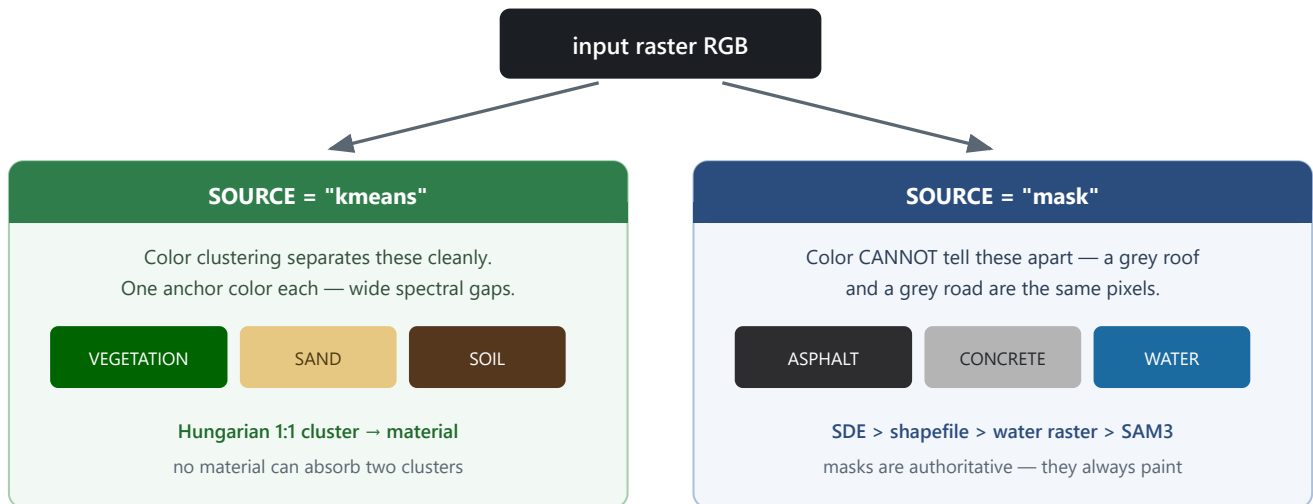
- `<stem>.tif` — the classified raster, EPSG:4326
- `<stem>.xml` — `<Composite_Material_Table>`, ARGB colors
- `<stem>.txr` / `.txs` — simulator sidecars

Why it isn't just clustering

Color alone can't identify a material. A grey roof, a grey road and a shadow are the same pixels. That single fact drives the entire architecture — see the next page.

Two sources of truth

Every material declares a `source`. Natural ground cover separates cleanly in color space, so it comes from KMeans. Built surfaces don't, so they come from a mask. Internalise this one split and most of the codebase stops being surprising.



Two consequences you will hit

Never mix mask classes into a shared KMeans

They have no useful spectral signature and they poison cluster assignment for everything else. This was a real production bug (`240c4ca`): batch outputs came back almost entirely water. `_split_classes_by_source()` exists purely to enforce it.

Masks are authoritative — they always paint

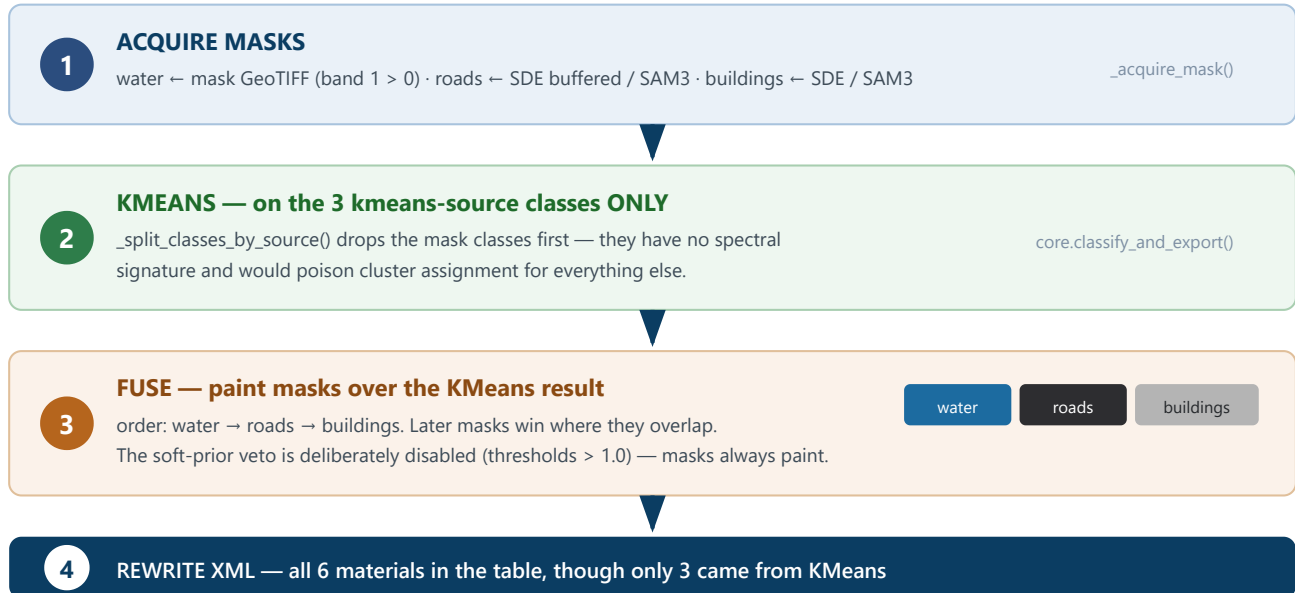
A soft-prior "veto" system exists in `pipeline.py`, but every threshold is set above `1.0`, which disables it. If a shapefile says a polygon is a building, it is painted concrete even if the pixels look green.

Anchor. Reference colors live in `shared/mea_defaults.json` — deliberately **one anchor per KMeans material**. Wide spectral separation makes a single anchor more robust than overlapping anchor clouds. Users can override them with the MEA Calibration Tool, which writes `%ProgramData%\MaterialClassification\mea_calibration_profile.json`.

classify_v6 — the orchestrator

Every entry point — web app, CLI, manifest mode — funnels into this one function in

`backend/app/pipeline.py`. Four phases, in order.



Hungarian assignment

Phase 2 uses strict 1:1 cluster→material matching rather than nearest-anchor, so one material can never absorb multiple clusters and starve the others.

Tiling

Big rasters are split into tiles and classified in a worker pool.
`suggest_tile_size()` picks the largest power-of-2 side that fits current free RAM.

Not padded

Outputs used to be padded to power-of-2 *dimensions*. Removed in `fd055ff` — it produced a black L-shaped strip on every output.

Phase 1 in detail — mask sources, in resolution order

SOURCE	CONFIGURED BY	NOTES
Water-mask GeoTIFF	<code>water_mask</code> in config, or <code>-water-mask</code>	Band 1 > 0 = water. Painted directly, no vector resolve.
Esri SDE	<code>sde</code> block in config	Buildings + roads via an <code>arcpy</code> subprocess under ArcGIS Pro's Python. Road lines buffered to polygons by a 3-tier width; bounds split into ≤5 km tiles to dodge the 2 GB shapefile cap.
Shapefiles	path arrays in config	Envelopes intersected with raster bounds (50 m buffer); only intersecting features are read, reprojected, unioned to a temp shp. Missing <code>.prj</code> → skipped.
SAM3	<code>--sam3-enabled</code> (default on)	Text-prompted segmentation. Falls back to OWLv2 + SAM2 when SAM3 weights are absent.

Four entry points, one pipeline

They differ only in how you feed them work. All four end up in `classify_v6`.

1 · Web app PRIMARY

React + Vite frontend, FastAPI backend. Pick a raster, tune materials, watch SSE progress on a Leaflet map.

```
# two terminals
.venv/Scripts/python.exe -m uvicorn \
  backend.app.main:app --port 8000
cd web_app && npm run dev # :5174
```

Gotcha: FastAPI also serves `web_app/dist/` at `:8000` — that's the *last built* frontend, not your live edits.

2 · CLI — file / folder

Same pipeline, scripted. The positional form auto-implies `--mea --sam3-enabled`.

```
cli.py <input.tif> <output.tif>
cli.py <in_folder> <out_folder>
```

Folder→folder is **batch mode**: one shared KMeans model across the whole folder, so tile boundaries don't show as color discontinuities. Trained on kmeans-source classes only.

3 · CLI — geocell manifest

One TOML = one OGC CDB geocell = one run. Built for the JARVIS automation. Detailed on the next page.

```
cli.py --manifest geocell.toml
```

Runnable example ships in the repo at `installer_assets/examples/sample_data/geocell.toml`.

4 · MEA Calibration Tool

Separate FastAPI + React app on port 8100. Sample real pixels from a raster to derive reference colors, write a profile, and the main app picks it up automatically.

The two apps are fully decoupled — the calibration tool only writes the profile JSON, the main app only reads it. No IPC.

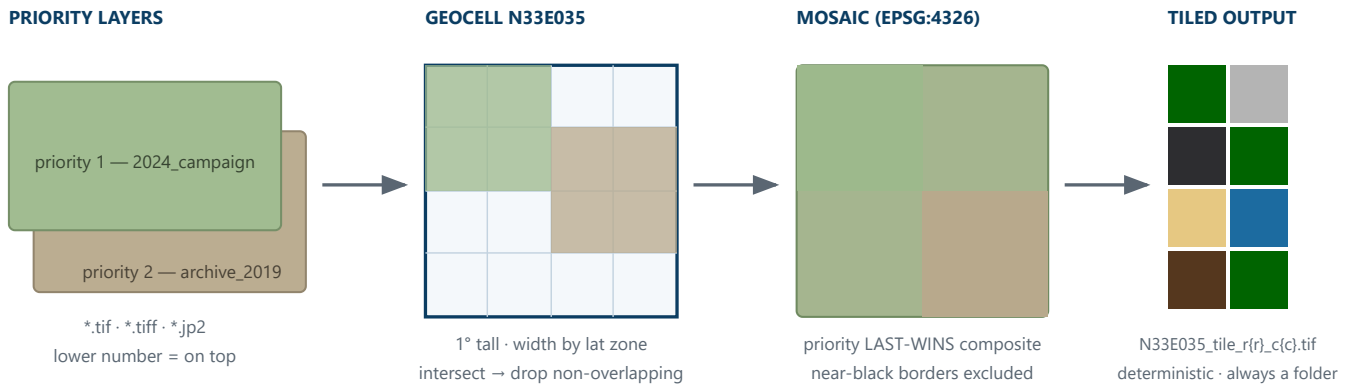
`tkinter_app.py` also exists and is unmaintained. It hasn't tracked the phase 4–6 MEA changes. Don't use it as a reference for current behaviour, and don't spend time on it.

What differs between them

	WEB APP	CLI FILE/FOLDER	CLI MANIFEST	CALIBRATION TOOL
Input	one raster, picked in the UI	file or folder	priority layers of a geocell	one raster, for sampling
Output shape	single raster	raster, or one per input	always a tile folder	a profile JSON
Shared KMeans	batch endpoint only	yes, in folder mode	n/a — one mosaic	n/a
Needs PyTorch	only for SAM3 / AI features	only with SAM3	no, if <code>sam3 = false</code>	no

The manifest path

The newest and least obvious mode. Instead of one input raster you declare **priority layers** of source imagery for one map cell; the tool composites them, then classifies the composite.



manifest.py → geocell.py → mosaic_catalog.py → mosaic_builder.py → pipeline.classify_v6()
 Tiling is FORCED at 512 px so the deliverable shape is deterministic regardless of the worker's free RAM.

A minimal manifest

```
[geocell]
south_lat = 33
west_lon  = 35

[[layers]]
folder    = "D:/orthos/2024_campaign"
priority  = 1          # 1 = highest

[[layers]]
folder    = "D:/orthos/archive_2019"
priority  = 2

[classify]          # optional
sam3 = false        # no PyTorch needed

[output]
path      = "D:/out/N33E035.tif"
overwrite = false
```

Design decisions that look like bugs

- **Output is always a folder**, never the `[output].path` file. Tiling is forced so the shape is deterministic; the overwrite guard therefore checks the `_classified_tiles/` directory.
- **Tile names derive from the cell name**, not the temp mosaic's stem — so the same manifest yields byte-identical filenames across runs.
- **Lower priority number = on top**, matching QGIS / Photoshop. Flat `sources` entries composite *below* every layer.
- **Unknown table = hard error**. Every model sets `extra="forbid"`, so a typo'd table name fails loudly instead of being silently ignored.

Known limit (Bug 4). The mosaic is built full-frame in RAM — ~7.35 GB measured for one cell. On a constrained box that's the ceiling. The windowed rewrite sits on the open PR #4 branch.

Where things live & what to read

Do not start with `core.py`. It is 4,538 lines and it is the implementation, not the design. Read in this order instead.

#	FILE	LINES	WHY
1	<code>backend/app/pipeline.py</code>	765	The orchestrator. Best single file for understanding the product.
2	<code>cli.py</code>	955	Every mode's entry logic, plus the embedded user guide (<code>--examples</code>).
3	<code>backend/app/main.py</code>	1,334	The HTTP surface + SSE progress / cancellation plumbing.
4	<code>backend/app/manifest.py</code>	255	Small and well-commented; shows the geocell design cleanly.
5	<code>backend/app/shapefile_resolver.py</code>	371	The "smart-trim" GIS resolver — a genuinely clever bit.
6	<code>backend/app/core.py</code>	4,538	Reference only. Grep it; don't read it front to back.

Module map

backend/app/

- `core.py` — KMeans engine, tiling, MEA export, XML/.txr/.txs writers
- `pipeline.py` — `classify_v6`
- `main.py` — FastAPI endpoints, SSE, cancellation
- `road_extraction.py` — OWLv2 + SAM2/3, `FEATURE_CONFIGS`
- `manifest.py` · `geocell.py` · `mosaic_catalog.py` · `mosaic_builder.py` — geocell mode
- `shapefile_config.py` · `shapefile_resolver.py` — GIS masks
- `sde_extractor.py` · `sde_arcpy_worker.py` — Esri SDE
- `config.py` · `mea_profile.py` — persisted settings

everything else

- `web_app/src/` — `App.tsx`, `api/client.ts`, `store/index.tsx` (reducer = UI truth), `components/sidebar/`
- `shared/mea_defaults.json` — the live anchors
- `tools/sync_mirrors.py` — mirror discipline
- `offline_installer/` — air-gapped payload + a full app mirror
- `installer_assets/` — files the pre-compiled installer exe predates
- `backend/tests/` — 9 modules, 84 tests
- `docs/` — topic deep-dives

Mirror discipline — the #1 way to break this repo. `offline_installer/app/{backend,web_app,shared}` is a **byte-identical mirror** of the dev tree, because the offline installer ships pre-built app files. Edit one side and not the other and the installed product silently differs from the code you tested. After editing a mirrored file run `tools/sync_mirrors.py`.

Get it running

The single most important environment rule: always use `.venv\Scripts\python.exe`. A bare `python` on a dev box here may resolve to a different install where `pkg_resources` is gone and the GroundingDINO / `triton-windows` chain breaks. Every command below spells out the interpreter on purpose.

1 Prerequisites. Python **3.11.x** (not 3.12+, not 3.14) — 3.11.9 is what the installer ships. Node 16+ only if you want the web UI. An NVIDIA GPU is optional; everything works CPU-only.

2 Create the venv and install.

```
py -3.11 -m venv .venv
.venv/Scripts/python.exe -m pip install -r backend/requirements.txt
# optional GPU pack (CuPy + CUDA 12.4-pinned runtime wheels)
.venv/Scripts/python.exe -m pip install -r backend/requirements-gpu.txt
# frontend, only if you want the web UI
cd web_app && npm install
```

3 Check which compute backend you got.

```
.venv/Scripts/python.exe -c "from backend.app import core; print(core._ACCEL_ENGINE)"
```

The probe order is `faiss-gpu` → `cupy` → `faiss-cpu` → `cuml` → `sklearn`. `faiss-cpu` or `sklearn` is a perfectly fine result — it just means no GPU pack. A broken CuPy is a warning, never a hard failure.

4 Run something real, on the sample data that ships in the repo.

```
.venv/Scripts/python.exe cli.py --examples      # embedded user guide
.venv/Scripts/python.exe cli.py --manifest \
  installer_assets/examples/sample_data/geocell.toml
```

5 Start the web app — two terminals, or just `start_webapp.bat`. Backend `:8000` (Swagger at `/docs`), frontend `:5174`.

Before you open a PR

- ☐ `.venv/Scripts/python.exe -m pip install pytest` — it is **not** in `requirements.txt`
- ☐ `.venv/Scripts/python.exe -m pytest backend/tests -q`
- ☐ `.venv/Scripts/python.exe tools/sync_mirrors.py --check`
- ☐ Branch → PR → merge. Conventional prefixes: `feat:` `fix:` `docs:` `perf:` `chore:`

There is no CI. `.github/` contains only `copilot-instructions.md` — no workflows. The mirror check and the test suite are described as "CI mode" throughout the docs, but nothing runs them automatically. Run both yourself.

Current state & landmines

Merged — the manifest/geocell hardening line

PR	WHAT IT DELIVERED	MERGED
#1	JPEG 2000 discovery alongside GeoTIFF in geocell layers	2026-07-09
#2	Optional <code>[classify]</code> table + tiled geocell output	2026-07-12
#3	Deterministic tile names + overwrite guard / clean	2026-07-13

Open — PR #4 `feat/silent-managed-installer`

Opened 2026-07-16, tip `f10889d`. 9 commits, ~3.7k added lines, 33 files. A clean fast-forward ahead of `main`. It bundles **two unrelated bodies of work**:

(a) JARVIS silent installer

`installer_silent/`, `publish.ps1`, `docs/AIRGAP_A4000_DEPLOY.md`. A headless installer built around a native MSVC stub — native because Windows can't launch a *managed* exe carrying a >4 GB overlay. The ~170 MB artifact is deliberately not standalone; ~15 GB of shared data is provisioned once per box.

(b) Tier 0–2 mosaic overhaul

Windowed, parallel, area-average geocell join; overview-pinned reads; footprint cache; resumable tiled mosaic cache with VRT handoff. Plus `benchmarks/`.

This is the fix for Bug 4 — the only open issue that makes a run fail outright rather than produce awkward output.

Known open bugs

#	ISSUE	STATUS
3	Per-tile <code>.xml</code> material tables aren't unified across a cell — index <i>N</i> can mean different materials in different tiles	Open. Blocks the CDB rm-layer consumer. Nobody is on it.
4	Mosaic built full-frame in RAM → OOM / thrash on constrained boxes, with no tiling fallback	Fix on PR #4 , not on <code>main</code> .

Things that are *supposed* to look wrong

- `shared/mea_classes.json` lists 13 classes. Leftover from before phase 4; **no code reads it**. Live schema = `MEA_CLASSES` in `core.py` + `shared/mea_defaults.json`. Don't wire it back in.
- Deleted wheels in `git status`**. The CUDA 12.9 wheels and a stray CPU torchvision were deliberately quarantined — 12.9 needs driver ≥ 575 and the target A4000 box reports CUDA 12.4, so they'd silently drop the pipeline to CPU.
- `graphify-out/` is **referenced but absent** — lost in the May-2026 deletion incident, tool not installed. And `mea.ts` marks `BM_WATER` as `"kmeans"` where the backend says `"mask"` (cosmetic; that field only labels the calibration UI).

Full detail: `docs/PROJECT_STATUS.md` and `docs/MC_MANIFEST_BUGS_2026-07-12.md`.

Where to go next

This deck is the overview. Everything below is the depth.

House style, and it is enforced in review: think before coding and state your assumptions; simplicity first — minimum code, no speculative abstractions; surgical changes — touch only what you must, no drive-by refactors; aggressive dead-code removal over backwards-compatible shims.

YOU WANT TO...	READ
Follow the full onboarding path in prose	<code>ONBOARDING.md</code>
Run something today	<code>RUNNING_GUIDE.md</code>
Use the CLI properly	<code>docs/CLI_GUIDE.md</code> · <code>cli.py --examples</code>
Understand the wiring	<code>docs/ARCHITECTURE.md</code>
Call the HTTP API	<code>docs/API_REFERENCE.md</code> · Swagger at <code>:8000/docs</code>
Know what's broken / in flight	<code>docs/PROJECT_STATUS.md</code>
Tune the AI masks	<code>docs/AI_FEATURE_EXTRACTION.md</code>
Get GPU working	<code>docs/GPU_ACCELERATION.md</code>
Understand tile mode	<code>docs/TILING.md</code>
Tune material colors	<code>docs/MEA_CALIBRATION_TOOL.md</code>
Deploy to an offline station	<code>STANDALONE_DEPLOYMENT.md</code>
Debug "0 pixels rasterized"	<code>RASTERIZE_DEBUG.md</code> (Hebrew)

Glossary

TERM	MEANING	TERM	MEANING
MEA	The material schema this targets. 6 classes, <code>BM_*</code> .	OWLv2	Open-vocab detector, paired with SAM2 as the SAM3 fallback.
Orthophoto	Top-down georeferenced aerial imagery, geometrically corrected.	SDE	Esri enterprise geodatabase. Read via an <code>arcpy</code> subprocess.
Geocell	OGC CDB unit: 1°-tall cell, widening toward the poles. <code>N33E035</code> .	JARVIS	The external orchestrator that drives manifest mode.
CDB	OGC Common DataBase — the terrain standard the output feeds.	Manifest	The TOML describing one geocell run.
GSD	Ground Sample Distance — real-world size of one pixel.	Hungarian	Optimal 1:1 matching; maps clusters to materials.
SAM3	Meta's Segment Anything 3. Supplies road / building masks.	Mirror	<code>offline_installer/app/</code> — byte-identical app copy.

Regenerate this deck: `.venv/Scripts/python.exe tools/build_onboarding_pdf.py`